# ENS 491 – Graduation Project (Design)

# Progress Report I

## Project Title: Multi-Agent Corporate Knowledge Intelligence System with Advanced RAG and Dynamic Graph Reasoning (MACKIS)


**Group Members:**

**Bugrahan Yapilmisev 32147**

**Ahmet Caliskan 31065**

**Murat Berke Türkan 30875**

**Supervisor(s):**

**İnanc Arin**


**Date:**

**04.01.2026**

## ABSTRACT

This project presents MACKIS, an intelligent Retrieval-Augmented Generation (RAG)–based chat assistant designed to support Sabancı University students and staff in accessing institutional knowledge. The system enables natural-language querying over official university documents that are publicly or internally available on university web platforms and used for academic and administrative purposes.

MACKIS addresses limitations of traditional search engines and chatbot systems by combining semantic retrieval with advanced query understanding mechanisms such as intent detection, follow-up awareness, negation handling, and document-level reasoning. The system is implemented as a modular, self-contained prototype using open-source technologies, consisting of a Python-based backend and a modern web-based frontend. This report describes the system design, technical developments achieved so far, encountered challenges, and the planned evaluation strategy, forming the foundation for further development in ENS 492.


## 1. PROJECT SUMMARY

### 1.1 Project Description

This project focuses on the design and development of MACKIS, an intelligent chat assistant tailored for Sabancı University students and staff. The system is designed to provide accurate, context-aware, and reliable answers to user queries by leveraging Retrieval-Augmented Generation (RAG) over official university documents that are available through university web pages and internal platforms for academic and administrative use.

These documents typically contain information related to academic procedures, student status, eligibility rules, and institutional policies, and are often distributed across different sections of university websites. Manually locating specific information within this corpus can be time-consuming and error-prone.

Unlike basic chatbot systems, MACKIS incorporates advanced query understanding mechanisms, including intent detection, follow-up awareness, negation handling, and document-level reasoning, in order to interpret user questions more accurately. The system architecture consists of a Python-based backend implemented with FastAPI, responsible for retrieval, ranking, and large language model orchestration, and a modern React-based frontend that provides an interactive and transparent user experience with explicit source citations.

### 1.2 Gap in Existing Solutions & Motivation

Currently, accessing institutional information at universities often requires navigating through numerous web pages and documents with varying structures and formats. Traditional keyword-based search engines rely heavily on lexical matching and fail to capture the semantic intent of user queries, frequently returning incomplete or loosely related results.

In addition, many existing chatbot solutions generate responses without strict grounding in authoritative sources. This can lead to hallucinated answers, outdated information, or ambiguity, which significantly reduces user trust particularly in an academic environment where accuracy is critical.

The motivation of this project is to address these limitations by building a retrieval-grounded, intent-aware knowledge assistant that ensures responses are derived strictly from official university sources and are transparently supported by citations.

### 1.3 Objectives & Intended Results

The main objectives of this project are to achieve high accuracy by generating answers strictly based on retrieved university documents, to ensure context awareness by correctly interpreting user intent, follow-up questions, and negations in multi-turn dialogue, to provide source transparency through document-level citations including document name, section or page, and confidence indicators, to enable user history management through secure user authentication and persistent access to previous conversations, and to design for scalability with a modular and configurable architecture that supports the addition of new documents and reasoning components.

The intended outcome is a robust prototype that demonstrates reliable institutional knowledge access while satisfying realistic engineering constraints such as data privacy, response latency, and accuracy thresholds. The development process follows fundamental engineering design elements, including analysis, synthesis, construction, testing, and evaluation.

## 2. SCIENTIFIC / TECHNICAL DEVELOPMENTS

### 2.1 Conceptual Design & System Architecture

To achieve a reliable university assistant, a layered system architecture was adopted. The conceptual design divides the system into three main layers.

The Data Layer manages both structured and unstructured data. Structured data such as users, conversations, and chat history are stored in PostgreSQL using Supabase as the cloud provider. Unstructured textual content is processed into vector embeddings and stored in ChromaDB using the BGE-M3 embedding model. The Application Logic Layer, implemented as a FastAPI-based backend service, orchestrates authentication, query understanding, semantic retrieval, reranking, and response generation. Finally, the Presentation Layer consists of a React-based responsive interface built with TypeScript. It manages user state and visualizes AI-generated answers together with their supporting sources.

This separation, visualized in the modular flow between components in Figure 1, ensures high maintainability and extensibility for future system enhancements, such as multi-agent reasoning.

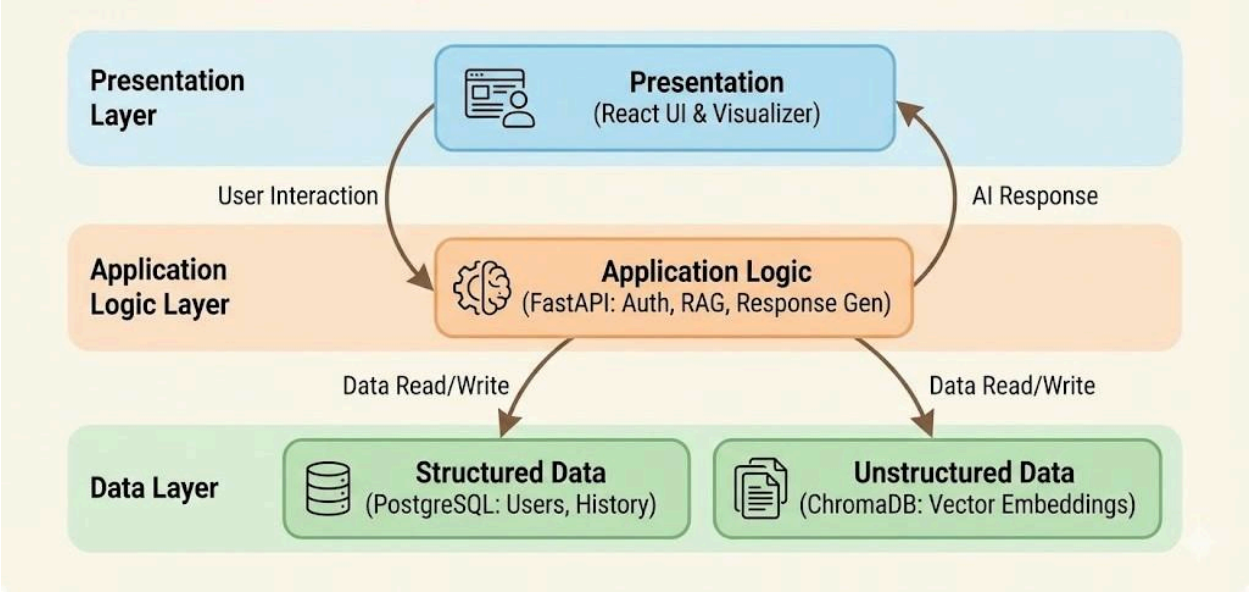


***Figure 1:*** *Conceptual System Architecture Flowchart (Source: Generated by Google Gemini, 2026).*

### 2.2 Design Decisions & Technology Stack

During the preliminary design phase, several key technologies and design choices were made. FastAPI was chosen as the backend framework for its asynchronous performance and clear API design. ChromaDB serves as the vector database for efficient semantic similarity search, configured with persistent storage and a dedicated collection for university documents. The frontend framework uses React with TypeScript to ensure type safety and component reusability, with Vite as the build tool for fast development iteration. JWT-based authentication was implemented for stateless and secure session management, with tokens stored in localStorage and automatically attached to requests via Axios interceptors. For language models, the system integrates with Ollama to use models like Llama 3.2 for answer generation and BGE-M3 for embeddings, with the BAAI/bge-reranker-v2-m3 cross-encoder for reranking. These choices were made to ensure scalability, robustness, and ease of experimentation.

### 2.3 Advanced RAG Pipeline Design & Implementation

Significant progress has been made in developing an advanced RAG pipeline that extends beyond a standard vector-search-based approach through the integration of multiple complementary query understanding and retrieval refinement techniques, which are described below.

Intent Detection: User queries are classified into listing, counting, descriptive, or general intents using both lexical pattern matching and LLM-based classification. For Turkish queries, the system recognizes patterns like "kaç tane" or "isimlerini listele" while for English it detects phrases like "how many" or "list all". This enables intent-aware retrieval and response

generation where the system adjusts both its retrieval strategy and its prompt templates based on the detected intent.

Follow-Up & Dialogue Awareness: The system employs a multi-layered approach to detect follow-up queries. It uses lexical cues to detect Turkish pronouns and follow-up phrases such as "bunu", "daha detaylı", or "devam et". It also performs semantic similarity comparison between the current query and recent conversation history using cosine similarity with a configurable threshold of 0.60. Additionally, an LLM-based dialogue classifier determines whether the query is a follow-up and suggests an anchor strategy. When a follow-up is detected, the system constructs an anchor query by combining the most relevant previous question with the current query, ensuring conversational context is preserved.

Negation-Aware Retrieval: The system implements a sophisticated negation handling mechanism that operates in two modes. The LLM-based approach uses a specialized prompt to extract explicitly negated terms from queries, while the heuristic fallback uses window-based rules around negation markers such as "değil", "hariç", "not", or "except". Importantly, the system only considers terms as negated if they do not appear elsewhere in the query in a positive context. Documents whose titles or headers match negated terms are penalized in the ranking with a configurable penalty factor of 0.3.

Semantic Tag-Based Routing: Each document in the corpus is assigned open-ended semantic tags during preprocessing using an LLM. At query time, the system infers query-level tags and performs fuzzy token-level matching between query tags and document tags. This approach improves document selection without relying on rigid keyword rules, allowing the system to route queries like "Erasmus başvurusu" to documents tagged with "erasmus_study" or "student_mobility".

Cross-Encoder Reranking & Confidence Filtering: Retrieved document chunks are reranked using the BGE-reranker-v2-m3 cross-encoder model. The system computes a hybrid score combining cross-encoder scores with vector similarity scores using a configurable weight of 0.7 for the cross-encoder. Additionally, a title overlap boost is applied when query terms appear in document titles. Results below a configurable CE score threshold of 0.70 are filtered out, with a minimum of 4 and a maximum of 12 chunks retained.

Document-Centric Context Construction: Retrieved chunks are grouped by document path, and the system limits chunks per document to prevent fragmented or contradictory context. The top 3 documents with up to 4 chunks each are selected. For queries with listing or counting intent, the system may retrieve all chunks from a single highly-relevant document to ensure comprehensive coverage. Finally, Maximal Marginal Relevance (MMR) selection is applied with a lambda parameter of 0.7 to balance relevance and diversity.

### 2.4 Frontend & Backend Implementation Progress

On the frontend, a transparent user interface was implemented using React components and the Tailwind CSS framework. The ChatMessage component renders AI responses using React Markdown with GitHub Flavored Markdown support, enabling formatted outputs that include

code blocks, tables, and links. The SourceCard component displays document citations in a collapsible section beneath each AI response, showing filename, relevance score, and content preview. The interface includes a ConversationSidebar for managing multiple chat sessions, a CategoryFilter for topic-based navigation, and a KnowledgeBaseStats component showing the scope of indexed documents. A centralized Axios instance with request and response interceptors ensures secure API communication, automatically attaching JWT tokens to requests and handling 401 unauthorized responses.

On the backend, a comprehensive data model was implemented using SQLAlchemy ORM with 15 interconnected tables. The User model supports multiple roles including undergrad, grad, alumni, staff, admin, and guest with status tracking. The Conversation model links to users and stores chat history with archived status support. The QuerySession and QueryEvent models enable full traceability of user interactions, while the Answer and ChatMessage models store generated responses with token counts. The Document and Chunk models manage the knowledge base with support for embeddings via pgvector, and the RetrievalHit and AnswerCitation models provide complete provenance tracking. This demonstrates system-level integration beyond isolated retrieval functionality.

### 2.5 Knowledge Graph Integration & Future Reasoning Support

While RAG provides strong document grounding, it lacks explicit relational reasoning. To address this, a Knowledge Graph component has been introduced as part of the system's future reasoning layer.

Conceptual Design: The Knowledge Graph represents entities such as academic programs, student roles, regulations, and eligibility conditions, along with relations such as "requires", "applies_to", and "regulated_by". Each entity and relation is traceable to its source document through the doc_id foreign key in the KGNode table.

Preliminary Implementation: An initial KG extraction pipeline has been implemented in the GraphService class. The service uses an LLM with a carefully crafted system prompt to extract subject-relation-object triples from document chunks in a controlled JSON schema. Each triple includes head and tail entities with their types, and the relation type in snake_case format. The implementation includes fault-tolerant database operations using nested transactions, deduplication of triples at the Python level, and a search function that can query the graph for entities and their relationships using SQL queries on the kg_nodes and kg_edges tables.

Planned Interaction with RAG: The planned hybrid architecture will combine RAG-based retrieval with graph-based reasoning. Retrieved documents will be mapped to graph entities, enabling constraint validation, multi-hop reasoning, and improved consistency for follow-up queries. The Knowledge Graph also serves as a foundation for future multi-agent extensions, where specialized agents can operate over shared graph and retrieval representations.

### 2.6 Freedom-to-Use (FTU) & Commercialization

The project relies exclusively on permissively licensed open-source technologies. FastAPI, React, ChromaDB, and Sentence Transformers are all available under MIT or Apache 2.0

licenses. The Ollama framework enables local deployment of language models without external API dependencies or licensing concerns. No Freedom-to-Use issues have been identified. While the current focus is academic, the modular architecture with clear separation between data, logic, and presentation layers supports future commercialization for other knowledge-intensive institutions.

## 3. TEST & EXPERIMENT DESIGN

### 3.1 Experiment Objectives

A comprehensive evaluation is planned for ENS 492. The primary objective is to measure Retrieval Accuracy, specifically whether the system retrieves the correct document chunks for a given query. This will be measured using Hit@k metrics, where k ranges from 1 to 10. The secondary objective is to evaluate Generation Faithfulness, meaning whether generated answers strictly adhere to retrieved content without introducing hallucinated information. This will be assessed through semantic similarity measures and human evaluation.

### 3.2 Ground Truth Dataset

A manually curated dataset of 50 to 100 question-answer pairs will be constructed from official university documents. Each entry will include the original question in Turkish or English, a verified ground truth answer extracted directly from source documents, the source document path and relevant section, and the expected intent classification. Questions will cover different intent types including listing, counting, descriptive, and general queries, as well as edge cases like negations and follow-up questions.

### 3.3 Experimental Procedure

The experiment will follow a systematic procedure. First, all test queries will be executed against the system in batch mode, with the RAG pipeline logging retrieved chunks, confidence scores, and generated answers. Second, for retrieval evaluation, the system will compute Hit@1, Hit@5, and Hit@10 by checking if the ground truth document appears in the top k retrieved results. Third, for generation evaluation, the system will compute semantic similarity between generated answers and ground truth using cosine similarity of sentence embeddings, and optionally use the RAGAS framework for automated evaluation of faithfulness and relevance. Fourth, for latency measurement, end-to-end response times will be recorded to ensure real-time usability with a target of under 10 seconds per query. Fifth, ablation studies will be conducted by disabling individual components such as cross-encoder reranking, negation handling, and tag-based routing to measure their individual contributions.

### 3.4 Expected Outcomes & Improvement Strategy

Based on preliminary testing during development, initial retrieval Hit@5 accuracy is expected to be approximately 75 to 80 percent. If performance is insufficient, improvements will focus on adjusting chunk size and overlap parameters which are currently set at 900 characters with 180

overlap, tuning the cross-encoder score threshold which is currently 0.70, implementing query expansion techniques for difficult queries, and enforcing stricter prompt constraints to reduce hallucinations in generated answers.

## 4. ENCOUNTERED PROBLEMS

During the development process, several challenges were encountered that necessitated adjustments to the original project plan. The initial architecture assumed a simpler retrieval pipeline, but early testing revealed that basic vector search produced too many irrelevant results, particularly for queries with implicit context or negations. This led to the decision to implement the more sophisticated multi-stage pipeline with cross-encoder reranking and tag-based routing, which required approximately two additional weeks of development.

Handling conversational context proved more complex than anticipated. Initial attempts to simply concatenate conversation history led to context pollution, where documents relevant to previous questions would dominate results for new questions. The solution was to implement the anchor query mechanism with semantic similarity-based selection of relevant history, which required careful tuning of similarity thresholds.

The negation handling component went through multiple iterations. The first heuristic-based approach was too aggressive, incorrectly penalizing documents that mentioned negated terms in any context. The refined approach using LLM-based extraction with heuristic fallback and the "only negated if not positive elsewhere" rule significantly improved precision.

Integration between the frontend and backend required addressing CORS configuration issues and ensuring consistent data formats between TypeScript interfaces and Python schemas. The decision to use Axios interceptors for token management simplified the frontend code but required careful handling of 401 responses.

In addition to the architectural and integration challenges, another difficulty arose from the size and diversity of the document corpus. After running the crawler on university web platforms, approximately 2,000 documents were collected. Some of these documents are outdated, replaced by newer versions, or no longer relevant to current academic procedures, which makes them harder to process and requires special handling during preprocessing and retrieval.

The large corpus also made development and testing more difficult. In early stages, it was not always clear how a query would interact with the dataset, making it harder to decide what to ask the model and what kind of response to expect. In addition, follow-up questions became more challenging because many documents contain overlapping but inconsistent information. To address this, the need for an internal corpus browsing tool became clear, allowing developers to inspect retrieved documents, understand why certain sources are selected, and better track how answers are generated across multi-turn interactions.

Despite these challenges, the original project goals remain relevant and achievable. The core objective of building a retrieval-grounded, context-aware assistant has been validated through

the current implementation. The adjustments made have actually strengthened the system's robustness and reduced technical debt that would have accumulated from simpler solutions.

According to the project timetable, the current status shows that document crawling and preprocessing is complete, the basic RAG pipeline is complete, the advanced retrieval mechanisms are complete, the frontend user interface is complete with ongoing refinements, the authentication and session management is complete, and the Knowledge Graph extraction pipeline is in a preliminary implementation stage. The project is approximately one week behind the original schedule due to the complexity of the advanced retrieval components, but this delay is being addressed through parallel development of the remaining features.

## 5. TASKS TO BE COMPLETED UNTIL PROGRESS REPORT II

The work planned for the next reporting period follows the capstone design phases and is organized into a structured timeline spanning approximately ten weeks.

During the first two weeks, corresponding to the Preliminary Design phase, the focus will be on stabilizing and validating the current system architecture. Configuration parameters related to retrieval, reranking, and prompt behavior will be systematically reviewed and refined to support controlled experimentation. Additional unit and integration tests will be added for critical components to ensure robustness and reproducibility.

During weeks three and four, corresponding to the Detailed Design phase, the Knowledge Graph storage and query interface will be finalized. Graph population will be extended to cover a broader portion of the document corpus, and consistency checks will be introduced to reduce noise in extracted relations. In parallel, the hybrid interaction between RAG-based retrieval and graph-based reasoning will be formally designed and prototyped.

During weeks five and six, corresponding to the Construction phase, multi-agent reasoning support will be implemented. This includes introducing specialized agents responsible for retrieval refinement, graph-based constraint checking, and dialogue management, along with an orchestration mechanism to coordinate their outputs. The citation generation mechanism will also be enhanced to provide more fine-grained references, such as section-level context where available.

During weeks seven and eight, corresponding to the Testing phase, systematic evaluation experiments will be conducted using the curated ground truth dataset. Component-level ablation studies will be performed to measure the individual contribution of major pipeline elements, including reranking, negation handling, and tag-based routing. In addition, user testing with a limited group of students and staff will be carried out to assess usability and answer clarity.

During weeks nine and ten, corresponding to the Evaluation phase, experimental results will be analyzed and documented. The system will be compared against baseline approaches such as simple vector similarity search and keyword-based retrieval. Based on these findings, final

refinements will be applied, and the project report and presentation materials for ENS 492 will be prepared.

## 6. REFERENCES


Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020.

Karpukhin, V., et al. (2020). "Dense Passage Retrieval for Open-Domain Question Answering." EMNLP 2020.

Nogueira, R., & Cho, K. (2019). "Passage Re-ranking with BERT." arXiv preprint arXiv:1901.04085.

Chen, Z., et al. (2024). "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." arXiv preprint arXiv:2402.03216.

FastAPI Documentation. https://fastapi.tiangolo.com/

ChromaDB Documentation. https://docs.trychroma.com/

React Documentation. https://react.dev/

Ollama Documentation. https://ollama.ai/

SQLAlchemy Documentation. https://docs.sqlalchemy.org/

Sentence Transformers Documentation. https://www.sbert.net/